

HDOC - Day 7

(C Programming)

C Programming Activity Report

Question 1 - Kinetic Energy of a Satellite

1. Problem Understanding, Input, Output and Formulae

Find the kinetic energy of a satellite when its mass and orbital velocity are given.

Inputs: mass (kg), orbital velocity (m/s).

Outputs: kinetic energy (J).

Formulae:

Kinetic Energy = $\frac{1}{2} \times \text{mass} \times \text{velocity} \times \text{velocity}$

2. Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	mass=2, velocity=3	9.00 J	9.00 J	PASS
2	mass=10, velocity=4	80.00 J	80.00 J	PASS
3	mass=5, velocity=2	10.00 J	10.00 J	PASS
4	mass=0, velocity=10	0.00 J	0.00 J	PASS

3. Algorithm and Evaluation

1. Start.
2. Declare mass, velocity and kineticEnergy.
3. Input mass and velocity.
4. Calculate kineticEnergy = $0.5 \times \text{mass} \times \text{velocity} \times \text{velocity}$.
5. Display kineticEnergy.
6. Stop.

Evaluation: The algorithm directly applies the kinetic-energy formula. The test cases include normal values and a zero-mass case.

4. C Program

```
#include <stdio.h>

int main()
{
    float mass, velocity, kineticEnergy;

    printf("Enter mass of satellite (kg): ");
    scanf("%f", &mass);

    printf("Enter orbital velocity (m/s): ");
    scanf("%f", &velocity);

    kineticEnergy = 0.5 * mass * velocity * velocity;

    printf("Kinetic Energy = %.2f J\n", kineticEnergy);

    return 0;
}
```

5. Compilation, Execution and Test Evaluation

Compilation Command:

```
gcc satellite.c -o satellite
```

Execution Command:

```
./satellite
```

6. Compilation and Execution

```
(kali㉿kali)-[~/hdoc7]
└─$ vi satellite.c

(kali㉿kali)-[~/hdoc7]
└─$ gcc satellite.c -o satellite
```

```
(kali㉿kali)-[~/hdoc7]
└─$ ./satellite
Enter mass of satellite: 10
Enter orbital velocity: 4
Kinetic Energy = 80.00 J
```

Question 2 - Frequency of AC

1. Problem Understanding, Input, Output and Formulae

Find the frequency of alternating current (AC) when its time period is given.

Inputs: time period T (seconds).

Outputs: frequency f (hertz).

Formulae:

Frequency = $1 / \text{Time Period}$

$f = 1 / T$

2. Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	T=0.02 s	50.00 Hz	50.00 Hz	PASS
2	T=0.01 s	100.00 Hz	100.00 Hz	PASS
3	T=0.04 s	25.00 Hz	25.00 Hz	PASS
4	T=0.5 s	2.00 Hz	2.00 Hz	PASS

3. Algorithm and Evaluation

1. Start.
2. Declare timePeriod and frequency.
3. Input timePeriod.
4. Calculate frequency = $1.0 / \text{timePeriod}$.
5. Display frequency.
6. Stop.

Evaluation: The algorithm calculates the reciprocal of the time period. The test cases use positive non-zero time periods because division by zero is undefined.

4. C Program

```
#include <stdio.h>

int main()
{
    float timePeriod, frequency;

    printf("Enter time period (s): ");
    scanf("%f", &timePeriod);

    frequency = 1.0 / timePeriod;

    printf("Frequency = %.2f Hz\n", frequency);

    return 0;
}
```

5. Compilation, Execution and Test Evaluation

Compilation Command:

```
gcc frequency.c -o frequency
```

Execution Command:

```
./frequency
```

6. Compilation and Execution

```
(kali㉿kali)-[~/hdoc7]  
$ vi frequency.c  
  
(kali㉿kali)-[~/hdoc7]  
$ gcc frequency.c -o frequency
```

```
(kali㉿kali)-[~/hdoc7]  
$ ./frequency  
Enter time period: 0.02  
Frequency = 50.00 Hz
```

Question 3 - Mean Value of Three Measurements

1. Problem Understanding, Input, Output and Formulae

Find the mean value of three given measurements.

Inputs: three measurements m1, m2 and m3.

Outputs: mean value.

Formulae:

$$\text{Mean} = (m1 + m2 + m3) / 3$$

2. Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	10, 20, 30	20.00	20.00	PASS
2	5, 10, 15	10.00	10.00	PASS
3	2.5, 3.5, 4.5	3.50	3.50	PASS
4	0, 0, 0	0.00	0.00	PASS

3. Algorithm and Evaluation

1. Start.
2. Declare m1, m2, m3 and mean.
3. Input m1, m2 and m3.
4. Calculate mean = (m1 + m2 + m3) / 3.0.
5. Display mean.
6. Stop.

Evaluation: The algorithm adds the three measurements and divides the total by 3. The test cases include integer, decimal and zero measurements.

4. C Program

```
#include <stdio.h>

int main()
{
    float m1, m2, m3, mean;

    printf("Enter three measurements: ");
    scanf("%f %f %f", &m1, &m2, &m3);

    mean = (m1 + m2 + m3) / 3.0;

    printf("Mean = %.2f\n", mean);

    return 0;
}
```

5. Compilation, Execution and Test Evaluation

Compilation Command:

```
gcc mean.c -o mean
```

Execution Command:

```
./mean
```

6. Compilation and Execution

```
(kali㉿kali)-[~/hdoc7]  
$ vi mean.c  
  
(kali㉿kali)-[~/hdoc7]  
$ gcc mean.c -o mean
```

```
(kali㉿kali)-[~/hdoc7]  
$ ./mean  
Enter three measurements: 10 20 30  
Mean = 20.00
```